

Master 2 Research Report

LLM-Enhanced Graph-Based Intrusion Detection

A Parallel GNN–LLM Framework with Bidirectional
Refinement

Ali Alzein

Supervised by Fouad Al Tfaily

CESI LINEACT / Lebanese University

2026

Contents

1	Introduction and Context	2
1.1	Why combine graph and language representations?	3
1.2	Research problem and questions	4
1.3	Objectives and contributions	5
1.4	Report structure	6
2	Related Work	8
2.1	Graph-based network intrusion detection	8
2.2	Language models over security text and network traffic	9
2.3	Fusing structural and semantic representations	9
2.4	GNN and LLM combinations in intrusion detection	11
2.5	Iterative GNN–LLM feedback	11
2.6	Positioning	13
3	Method	15
3.1	Datasets and Common Processing Pipeline	15
3.2	Directed Multigraph Construction	15
3.3	Edge Feature Encoding	16
3.4	Structural Branch	17
3.5	Semantic Branch	18
3.6	Adaptive Gated Attention Fusion	19
3.7	Uncertainty-Guided Semantic Feedback	19
3.8	Baseline Protocol	21
3.9	Evaluation and Training Protocol	22

1 Introduction and Context

Intrusion detection is an operational problem as well as a classification problem. Missed attacks can leave harmful activity unchallenged, while false alarms consume analyst attention and make genuine incidents harder to prioritise. Practical guidance therefore treats alert validation, false-positive identification, and threat prioritisation as central parts of intrusion monitoring [19].

Modern networks also produce a continuous stream of connections whose meaning is rarely contained in a single record. A destination port, a packet count, or a flow duration can be informative, but the surrounding communication pattern often matters just as much. A distributed denial-of-service attack may appear as many sources converging on one destination, whereas a scan may appear as one source contacting many hosts or services. This relational character makes network traffic a natural candidate for graph-based analysis. At the same time, network events also carry semantic cues: protocol and port combinations, traffic magnitudes, and recurring behavioural descriptions can suggest what an interaction represents. An intrusion detection system that sees only one of these views is therefore working with an incomplete account of the traffic.

This report investigates whether structural and semantic evidence can be combined in a single edge-level intrusion detection framework using NF-UNSW-NB15 and NF-ToN-IoT. One experimental boundary must be stated from the outset: the current graph construction aggregates flows by source IP, destination IP, and attack class. Ground-truth labels therefore participate in defining edge identity, even though the attack name is excluded from the model’s input features and generated text. The consequence is explicit: the resulting scores compare representations and interaction mechanisms under a controlled protocol, but they are not deployment-valid estimates of detection performance on unseen, unlabelled traffic.

NF-UNSW-NB15 derives from UNSW-NB15, whose original benchmark was created from a mixture of normal activity and contemporary attacks in the UNSW Canberra Cyber Range Laboratory [16]. NF-ToN-IoT derives from the TON_IoT datasets and provides an IoT-focused telemetry setting [15]. This study uses both in their NetFlow-standardised forms [18]. One identical processing and modelling pipeline is applied to the two datasets. Every architectural choice is shared between them; only two quantities selected on validation data differ.

The two datasets were chosen because they occupy very different points on the class-imbalance axis. After aggregation, NF-UNSW-NB15 contains 656 edges, is 47.4% benign, and has a most-to-least-frequent class ratio of 12.4:1. Eight of its nine attack classes contain 40 edges and the ninth contains 25, so just one falls at or below that threshold. NF-ToN-IoT contains 2,127 edges, is 82.1% benign, and has a most-to-least-

frequent class ratio of 582:1. Seven of its nine attack classes contain 35 edges or fewer. Its distribution is far more skewed, with benign traffic dominant and most attack classes represented by a few dozen edges or fewer. By the most-to-least class ratio, the two graphs differ by roughly a factor of 47 in imbalance severity.

That difference is the methodological reason for using two datasets. A single dataset cannot separate a property of the architecture from a property of the class distribution that architecture happened to face. Holding the architecture fixed while changing the imbalance regime by roughly $47\times$ makes that distinction experimentally addressable. RQ4 already asks how class imbalance affects fusion and feedback; the two-dataset design gives that question two substantially different regimes to compare. No stage is assumed in advance to be sensitive or insensitive to that change.

The scoring protocol also differs in one necessary respect. Macro-F1 covers ten classes on NF-UNSW-NB15 and eight classes on NF-ToN-IoT, because two NF-ToN-IoT classes are too small to survive five-fold splitting. Their edges remain in the graph for message passing but are excluded from the metric. Because the class counts differ, absolute scores are not comparable across the two datasets; only the shape of the model ladder is. This constraint applies throughout the report.

1.1 Why combine graph and language representations?

Graph neural networks (GNNs) provide a direct way to represent communication structure. In this work, IP addresses form the nodes of a directed graph and aggregated flows form labelled edges. A graph-attention encoder propagates information between communicating hosts and produces a structural representation for each edge. This branch can learn patterns that are difficult to express as independent rows in a conventional feature table. It does not, however, possess an explicit vocabulary for interpreting the traffic. The graph tells the model *who communicated with whom* and how that interaction is positioned in the network; it does not automatically explain what a high-frequency TCP connection to a particular service may signify.

A pretrained cybersecurity language encoder offers a complementary view. The same aggregated flow can be written as a short, controlled sentence describing its endpoints, protocol, port, frequency, volume, and duration. The implementation uses CySecBERT, a BERT-class model adapted to cybersecurity text and evaluated by its authors on domain-specific representation and classification tasks [3]. It is frozen and used purely as a sentence encoder. Throughout this report, this component is called the *semantic branch*. That terminology distinguishes its representational role from that of a generative large language model.

Neither representation is assumed to be superior in every case. The central idea is

instead to let them challenge and complement one another. The parallel fusion stage runs the graph and semantic encoders side by side and combines their edge embeddings through Adaptive Gated Attention Fusion (AGAF). The feedback stage adds uncertainty-guided semantic feedback: edges with high-entropy GNN predictions are selected for semantic consultation, and the resulting signal is reintroduced by modifying the edge representations that the next graph update consumes. A second design, which biases graph attention directly, was also implemented and measured; it produced no change in prediction and is reported as a verified negative result.

The coupling is bidirectional in routing. The graph model’s per-edge uncertainty decides which edges are consulted, while the semantic branch returns a per-edge judgement that re-enters the edge representations consumed by the next graph update. It is not bidirectional in content: the consultant’s judgement depends on that edge’s text alone, not on the graph model’s current belief about the edge.

This general dual-stream idea is not novel by itself. Parallel BERT and graph encoders with attention-based fusion have already been studied outside network security, for example in misinformation detection [14]. Work in network intrusion detection has also combined GNNs and generative language models for a different role: XG-NID performs intrusion classification with a heterogeneous GNN and uses an LLM after classification to generate explanations and remedial guidance [6]. DAS is the closest prior work to the feedback mechanism considered here. It already implements a closed feedback loop between a frozen GNN and an LLM, feeds refined semantics back to update the same graph learner, and constructs node descriptions by expressing structural statistics in natural language; its setting is node-level and concerns general text-rich graphs [21]. The contribution here is therefore instantiation plus measurement: an edge-level network-intrusion-detection instantiation, together with a diagnostic question about whether a constrained semantic consultant exploits a feedback channel that has the capacity to influence the next graph update. Within the literature reviewed here, we are not aware of prior work combining bidirectional iterative coupling between a graph model and a semantic branch, joint attention-based fusion for the final classification, and edge-level network intrusion detection.

1.2 Research problem and questions

The research problem is not simply whether adding a language embedding raises a score. If one branch is already a strong classifier, a fusion module can appear successful while doing little more than copying that branch. Likewise, a feedback loop can improve the final output because semantic features are fused at the classifier, even when the claimed attention re-entry mechanism contributes nothing. The evaluation must therefore sep-

arate the value of the two modalities, the fusion mechanism, the uncertainty-selection policy, and the iterative update itself.

The study is organised around five research questions:

- RQ1.** How do the structural GNN branch and the semantic branch perform individually on NF-UNSW-NB15 and NF-ToN-IoT under the same five-fold out-of-fold evaluation protocol?
- RQ2.** Does AGAF outperform both single-branch baselines on NF-UNSW-NB15 and NF-ToN-IoT, and are the observed differences supported by uncertainty estimates and robust comparisons?
- RQ3.** Does uncertainty-guided semantic feedback add value beyond static fusion on NF-UNSW-NB15 and NF-ToN-IoT? If so, does the gain come from semantic re-entry into the graph representation or from final-output fusion?
- RQ4.** How do graph topology, repeated endpoint pairs, class imbalance, and the strength of the semantic consultant affect fusion and feedback on NF-UNSW-NB15 and NF-ToN-IoT?
- RQ5.** How does the feedback loop compare against published graph-based intrusion detection architectures re-trained on the same aggregated representation under the same protocol on NF-UNSW-NB15 and NF-ToN-IoT, and how much of any observed difference is attributable to representation and tuning rather than to architecture?

These questions deliberately allow a negative answer. The purpose of the experiments is to determine when the proposed interaction works, when it does not, and why. This matters because a model ladder—GNN, semantic encoder, fusion, then feedback—is only meaningful if each higher rung is compared fairly with the strongest relevant lower rung.

1.3 Objectives and contributions

The practical objective is to build one reproducible pipeline that transforms NF-UNSW-NB15 and NF-ToN-IoT NetFlow records into two aligned views and evaluates every stage under one consistent protocol. The work makes five contributions.

1. It implements a directed, edge-classification graph pipeline in which flows are aggregated by source IP, destination IP, and attack class, with centrality-based node features and statistical edge attributes.

2. It constructs a corresponding internal knowledge representation and serialises each edge as controlled natural language. The text supplied to the encoder is kept label-free by an explicit validation guard, preventing the semantic branch from seeing the target attack name or a synonymous relation label.
3. It implements and compares two multimodal stages: AGAF for parallel structural–semantic fusion, and uncertainty-guided semantic feedback that reintroduces semantic evidence into the edge representations of uncertain edges.
4. It provides a controlled evaluation on NF-UNSW-NB15 and NF-ToN-IoT, including unimodal baselines, fusion comparisons, uncertainty-selection ablations, trained-head baselines, mechanism-isolation tests, and two published graph-based intrusion detection architectures re-trained on the same representation. It reports both positive and negative results to show which components carry the measured gain.
5. It provides a cross-dataset comparison under one shared architecture across two substantially different class-imbalance regimes, testing whether the behaviour of each stage in the model ladder is a property of the method or of the class distribution it faces.

The contribution is therefore not presented as a universal improvement over existing intrusion detectors. It is an implementation and controlled evaluation of a particular form of structural–semantic cooperation, together with evidence about its limits. Deployment-valid evaluation remains open and requires graph construction in which edge identity can be determined without access to the target class.

1.4 Report structure

The report is organised into five sections. This Introduction defines the problem, experimental scope, research questions, and contribution. Section 2 reviews graph-based intrusion detection, cybersecurity language encoders, structural–semantic fusion, and iterative graph–language feedback, and positions the study against the closest prior mechanisms. Section 3 describes the two datasets, graph and text construction, the structural and semantic branches, AGAF, uncertainty-guided semantic feedback, the baseline protocol, and the shared evaluation procedure.

The Results section reports the predefined model ladder, baseline comparisons, ablations, and mechanism-isolation measurements for both NF-UNSW-NB15 and NF-ToN-IoT, while keeping the different scored class sets explicit. The Discussion section interprets those measurements against the research questions, including the cross-dataset

imbalance comparison and the distinction between a feedback channel having capacity and a constrained semantic consultant making use of that channel. It also returns to the controlled-protocol limitation and the work still required for deployment-valid intrusion detection.

2 Related Work

2.1 Graph-based network intrusion detection

Intrusion detection is conventionally divided into signature-based and anomaly-based approaches [19], but either can be posed over a flow graph, and a flow graph makes the edge the natural prediction unit when the object to be classified is a communication event between two endpoints. Nodes represent communicating entities such as IP addresses; edges represent flows or aggregated flows. This formulation preserves the relational context around a connection without changing the classification target: the decision remains edge-level. It also separates information attached to the communicating endpoints from information carried by the flow itself.

E-GraphSAGE is an important reference point for this formulation. It applies a GNN to intrusion detection and classifies an edge from the concatenation of the embeddings of its two endpoints [12]. The edge’s own features do not reach the classifier. That choice gives the classification head a purely endpoint-derived view of the connection. A direct consequence is that two parallel edges with the same ordered endpoints present the same pair of endpoint embeddings to the edge classifier, even if the edges differ in attributes that are not otherwise represented at the head. The architecture can express structural context around an endpoint pair, but its final edge representation does not preserve an explicit channel for the attributes of the particular edge being classified.

TE-G-SAGE moves to an edge-aware GraphSAGE formulation: its supervised edge head consumes the edge’s own features, and the model is paired with SHAP-based explanation [?]. This changes the representational question. Once the edge itself is allowed to carry predictive information, the model can distinguish endpoint context from flow-specific attributes rather than requiring the classifier to infer the latter indirectly from node embeddings.

For network-flow intrusion detection that distinction matters, because the graph and the individual connection encode different kinds of evidence. The graph describes who communicates with whom and under what relational context; the edge describes the flow being classified. The graph-based IDS literature therefore motivates an edge-level representation retaining both endpoint structure and edge-specific content. The next question is whether edge content has to remain numeric, or whether some of it can be represented semantically.

2.2 Language models over security text and network traffic

Cybersecurity language encoders provide one route to domain-specific semantics. Cy-SecBERT is a BERT-class encoder adapted to cybersecurity text [3]. Its relevance here is representational: a pretrained cybersecurity language encoder maps controlled security-related text into a dense embedding without requiring the graph encoder to interpret textual tokens itself.

A separate line of work treats traffic as a language-like sequence. BSTFNet applies a BERT-class model, ET-BERT, to serialised packet-level traffic and combines that representation with a spatiotemporal branch for encrypted-traffic classification [8]. It uses no graph encoder. The precedent that matters is the serialisation step itself: network observations that begin as structured traffic fields can be converted into a controlled token sequence and processed by a BERT-class encoder.

These two lines solve different problems. Domain adaptation addresses the vocabulary and semantics of cybersecurity text; traffic serialisation creates text-like input from network measurements. Together they support a semantic branch in which a flow description is encoded as text rather than treating every field as a scalar magnitude. They do not, by themselves, determine how that semantic representation should interact with graph structure.

That interaction is the central fusion problem. Once a system holds one representation derived from topology and another derived from text, the design space is no longer only about which encoder to choose. It is about where the two modalities meet, and whether their relative influence is fixed across samples or allowed to change from one sample to the next.

2.3 Fusing structural and semantic representations

Fusion methods organise along two related axes: how tightly the modalities interact, and whether the fusion rule assigns the same modality weighting to every sample or adapts that weighting to the input. The second axis is the more important one here. Four recurring families cover the relevant design space.

The simplest family is late fusion with a fixed interface between branches. GMLM uses separate graph and language towers, concatenates their representations, and passes the result to an MLP [20]; its own comparison explicitly distinguishes this design from cross-attention, gating, and learned modality weights. BertGCN gives an even cleaner example of fixed weighting: its final prediction linearly interpolates a GCN prediction and a BERT prediction using a global hyperparameter [11]. Every sample receives the same interpolation coefficient. These methods keep the encoders modular, but the

fusion rule never decides that one modality should matter more for one input and less for another.

A second family introduces input-dependent gates. The Gated Multimodal Unit uses multiplicative gates to determine how each modality contributes to the fused activation [2]. This is the canonical primitive for sample-adaptive modality weighting. A graph-and-text instance appears in the Dual-Stream Graph-Augmented Transformer for fake-news detection, where BERT runs in parallel with a GAT and Graph Transformer branch and a trainable scoring function emits a scalar weight for each modality for each input [14]. Mechanically this is the closest analogue in the literature surveyed here to a fusion stage whose balance changes per sample.

A third family uses attention to let the modality representations interact directly. RAGFormer stacks the two branch embeddings as a short token sequence and applies self-attention over them with a residual connection [10]. CAST uses cross-attention between graph and text representations in materials prediction and motivates the text branch by arguing that graph conversion can discard global information which textual descriptions restore [9]; the semantic representation is therefore not a duplicate encoding of graph structure. BiGTex goes further and interleaves a two-way exchange: structural information enters the language model as a soft prompt, and text returns to the graph side through cross-attention with the query supplied from the graph side [4]. Attention-based interaction differs from global interpolation because its interaction weights are derived from the representations of the current input.

The fourth family removes a distinct late-fusion stage altogether by interleaving structural and semantic processing throughout the encoder. DET uses exactly the terms “structural encoder” and “semantic encoder” and argues against late fusion of separate encoders, preferring interleaved interaction [7]. GL-Fusion sits at the opposite end of the coupling spectrum from independent towers: message passing is folded into every transformer layer, leaving no separate fusion module [25]. These designs mix the modalities repeatedly rather than preserving two independent representations until classification.

The taxonomy separates two questions that are often collapsed. One is where graph and text meet: after separate encoding, inside a dedicated attention mechanism, or repeatedly throughout the encoder. The other is whether the relative influence of the modalities is globally fixed or varies with the sample. The second distinction becomes sharper on moving from general graph-text systems to intrusion detection, where the available GNN-language combinations tend to assign the two components sequential roles rather than fusing their representations.

2.4 GNN and LLM combinations in intrusion detection

Within the intrusion-detection works surveyed here, graph models and language models are combined sequentially rather than fused as two representations of the same edge-level decision. XG-NID is the clearest example. Its heterogeneous GNN performs the intrusion classification, and an LLM is used only after prediction, to produce human-readable explanations [6]. The system’s “dual modality” refers to flow-level and packet-level data inside the graph pipeline, not to graph and text embeddings fused for classification.

Agent-based analysis for GNN robustness reverses the order. An LLM agent analyses textual descriptions of node interactions and flags, removes, or reweights suspicious nodes and edges before the GNN processes the graph [26]. The language component acts as a preprocessing filter: its output changes the graph presented to the GNN, but the two components are never combined as concurrent representations at the classifier.

Other intrusion-detection systems occupy adjacent points in the design space without forming a GNN–language pair at all. GCN-2-Former combines a GCN with a temporal transformer and fuses the two by global average pooling and concatenation [1]; the second tower is a temporal transformer, not a language model. CPS-IDS uses gated fusion between a DeBERTa payload-semantic branch and a statistical branch [24], so it has both a language-based semantic component and an explicit fusion mechanism, but no graph encoder.

The distinction is functional rather than terminological. In this literature the language model either explains a GNN output after classification or alters graph inputs before message passing. The parallel-fusion examples occupy nearby configurations but either replace the language model with a non-language temporal transformer or replace the graph encoder with a statistical branch. Representation-level fusion and graph–language feedback therefore have to be examined in the broader graph-learning literature rather than inferred from the intrusion-detection examples alone.

2.5 Iterative GNN–LLM feedback

The broader GNN–LLM literature already contains the feedback mechanisms this report builds on. They are prior work and are not claimed here as contributions.

DAS is the closest prior work to the feedback mechanism used in this report. It implements an explicit closed feedback loop between a frozen GNN and an LLM: GNN predictions provide implicit supervision that steers semantic refinement, and the refined semantics are fed back to update the same graph learner [21]. DAS also constructs node descriptions by expressing structural statistics in natural language — degree, between-

ness, closeness, clustering coefficient, and square clustering coefficient — and gives a Majorization–Minimization argument under which its refinement loop monotonically decreases a task-adaptive surrogate objective. Its setting is node-level, over general text-rich graphs. Closed graph–language feedback, and the conversion of structural statistics into natural-language descriptions, therefore already exist in the literature.

LOGIN already couples uncertainty selection to language-model consultation. It uses MC-dropout variance to identify uncertain nodes, prompts the LLM with node text and one-hop topology, and receives a predicted label together with a free-text explanation [17]. Incorporation depends on correctness: if the LLM’s label matches the ground-truth label, the explanation is appended to the node text and the augmented text is re-embedded; if the label is wrong, edges in the node’s one-hop neighbourhood are pruned. The LLM remains outside autograd and edits the learning inputs between training rounds. Selecting uncertain graph elements for semantic consultation is therefore also established.

GLANCE makes the decision to consult the language model itself learnable. A lightweight router decides per node whether querying is worthwhile from inexpensive node features, and an advantage-based objective trains that router on the outcome of querying versus not querying, because the LLM call is non-differentiable [13]. Its gains concentrate on heterophilous and structurally ambiguous nodes. This addresses the same allocation problem as uncertainty-based consultation — directing language-model effort at selected graph elements rather than applying it uniformly — with a learned criterion in place of a fixed uncertainty rule.

RoGRAD places iterative refinement, rather than static one-shot signal injection, at the centre of its formulation [23]. It studies that refinement under graph deficiencies including noise, sparsity, and label scarcity, and its abstract claims priority for moving language-model augmentation of graphs from static injection to an iterative refinement paradigm. Iterative semantic refinement is therefore already an explicit contribution claim in the prior literature.

These four anchors settle the component question plainly. Closed GNN–language feedback already exists. Iterative semantic refinement already exists. Structural statistics have already been expressed as natural-language input. Uncertainty and learned routing have already been used to decide which graph elements receive language-model consultation. Taken component by component, the feedback mechanism is not the contribution of this report.

2.6 Positioning

The contribution is an edge-level intrusion-detection instantiation, and a measurement question about how such a feedback channel is used. The comparison therefore has to be made at the level of configuration rather than of individual mechanisms.

The four feedback anchors operate at node level, whereas the present setting treats a network flow as an edge and makes the classification decision there. Their semantic consultants are prompted generative LLMs; here the semantic branch is a frozen sentence encoder whose embeddings are scored against class prototypes. Their feedback actions refine semantic inputs or edit graph inputs; here the accepted semantic signal modifies the edge representations consumed by the next graph update. These are differences of instantiation, not claims that the underlying ideas originated here [13, 17, 21, 23].

Two signals travel in opposite directions in the resulting loop, and it is worth stating precisely which. The graph model’s per-edge uncertainty determines *which* edges are routed to the semantic branch, and that selection is recomputed as predictions shift; the semantic branch returns a per-edge judgement that re-enters the graph model’s edge representations. The coupling is bidirectional in this routing sense. It is not bidirectional in content: the consultant’s verdict on an edge depends on that edge’s text alone, not on what the graph model currently believes about it.

The final classification stage adds a further dimension to the configuration. Structural and semantic edge representations are combined through joint attention-based fusion with sample-dependent modality weighting, rather than assigning the semantic component a preprocessing or post-hoc role. Sample-adaptive fusion is itself established outside this setting: gated multimodal units provide input-dependent modality control, and the Dual-Stream Graph-Augmented Transformer assigns per-input scalar modality weights to parallel text and graph representations [2, 14]. Representation-level graph-text interaction is likewise established, through self-attention, cross-attention, and interleaved exchange [4, 9, 10]. None of those mechanisms is a contribution claim here.

Within the limits of the literature search represented by this survey, we are not aware of prior work combining all three of the following in one system: bidirectional iterative coupling between a graph model and a semantic branch, joint attention-based fusion for the final classification, and edge-level network intrusion detection. This is a statement about the configuration space our search covered. It does not imply a performance advantage, and it does not change the prior-art status of any individual component.

That positioning leaves a narrower research question. Existing feedback work asks whether a language-model-driven refinement process improves the graph-learning task, with the model revising semantics, modifying inputs, or selectively augmenting graph

elements [13, 17, 21, 23]. The present work instead instantiates the loop in network-flow intrusion detection with a deliberately more constrained consultant, and asks a diagnostic question: whether such a consultant exploits a feedback channel that has the capacity to influence subsequent graph updates. That question is underexplored in the work surveyed here. No claim about its answer belongs in this section.

The contribution is best described as instantiation plus measurement. The system instantiates bidirectional feedback and sample-adaptive structural–semantic fusion for edge-level network intrusion detection, then measures whether the constrained semantic consultant makes use of the channel it is given. It does not rest on inventing GNN–LLM feedback loops, iterative semantic refinement, natural-language descriptions of graph statistics, or selective semantic consultation. Those mechanisms have clear precedents. The point of this configuration is to make their interaction measurable in an edge-level network-IDS setting.

3 Method

3.1 Datasets and Common Processing Pipeline

We evaluate on NF-UNSW-NB15 and NF-ToN-IoT, the NetFlow-standardised variants of their respective intrusion-detection corpora [16, 18]. One pipeline processes both, and every architectural choice is shared between them. Only two quantities are permitted to differ, and both are selected on validation folds: the uncertainty threshold k and the feedback injection scale. This restriction is deliberate. A cross-dataset comparison in which the two ladders use different components measures the components, not the datasets.

The pipeline converts flow records into a graph, derives node and edge attributes, trains an edge-level structural model, builds label-free semantic representations for the same edges, and combines the two modalities through adaptive fusion. Every classification decision is edge-level: a flow, or an aggregate of flows sharing a grouping key, is an edge, never a node.

3.2 Directed Multigraph Construction

Flows are aggregated into a directed multigraph keyed on

(source IP, destination IP, attack type).

Parallel edges therefore exist between an ordered IP pair whenever those flows carry different attack types. Nodes are the distinct IP addresses. The resulting graphs are small: NF-UNSW-NB15 yields 49 nodes and 656 edges, NF-ToN-IoT 1,501 nodes and 2,127 edges.

Each node carries ten centrality columns: betweenness, PageRank, degree, closeness, eigenvector, k -core, k -truss, global betweenness, global PageRank, and modularity vitality. These are computed on an unweighted IP-to-IP graph with self-loops removed. Betweenness (normalised), PageRank, closeness and eigenvector centrality are computed on the directed graph; degree is *in-degree* centrality; k -core and k -truss are computed on the undirected projection. Modularity vitality is the leave-one-node-out drop in modularity under a greedy modularity partition of the undirected projection. Eigenvector centrality and the truss decomposition fall back to constants when they fail to converge. A node’s feature vector is the mean of these values over all incident edges, taking the node in both its source and destination roles.

One property of this feature set should be stated rather than left for a reader to

discover. On NF-UNSW-NB15 the “global” variants are populated from the same computation as their local counterparts, so `global_betweenness` and `global_pagerank` are exact duplicates of `betweenness` and `pagerank`; the ten columns span eight distinct measures. On NF-ToN-IoT, whose centralities were computed upstream, the global variants differ from the local ones. We report ten input channels because that is what the model receives, but the effective rank of the node features is lower on NF-UNSW-NB15.

Each edge carries five attributes. Three are numeric: the number of constituent flows (`flow_count`), their summed byte count (`total_bytes`), and their mean duration (`avg_duration`). Two are categorical: the modal protocol and the modal port, with ties broken toward the smaller value.

Because attack type is part of the aggregation key, the label participates in defining edge identity. This is a property of the protocol, not a bookkeeping detail. Two flows between the same IP pair that differ only in their label become two distinct edges, and a deployed system observing unlabelled traffic could not perform this separation. The scores reported in this work therefore compare representations and mechanisms under a controlled protocol. They are not deployment-valid detection estimates, and we do not present them as such.

3.3 Edge Feature Encoding

The encoding separates quantities from identifiers. Columns 0–2 receive a $\log(1 + x)$ transform, then Z-scoring. Columns 3–4 are rewritten as contiguous vocabulary indices and consumed by learned embedding tables of width 4 (protocol) and 32 (port). Concatenation gives a $3 + 4 + 32 = 39$ -dimensional encoded edge vector. The vocabularies are small: 7 protocols and 169 ports on NF-UNSW-NB15, 4 and 235 on NF-ToN-IoT.

This corrects a specific defect in an earlier version of the pipeline, which passed protocol and port through as raw magnitudes. Port then entered the network at roughly $11,000\times$ the scale of every normalised column, and the ordering it imposed is meaningless: port 443 is not “more” than port 80. The embedding tables let the model represent protocol and port identity without inheriting the arithmetic of their indices.

The Z-score statistics and the two vocabularies are fitted once over all edges, before the cross-validation split. This is transductive, and we flag it explicitly. It is standard for a single fixed graph, and it transfers no label information, but it is not the same as fitting scalers inside each training fold.

Table 1: Per-variant edge focus weights. Applied as a fixed diagonal scaling of the encoded edge vector; the protocol and port weights are broadcast across their embedding blocks.

Variant	flow_count	total_bytes	avg_duration	protocol	port
temporal	1.25	0.75	1.50	0.50	0.25
content	0.50	1.50	0.50	1.00	1.25
behavioural	1.25	0.75	1.00	0.75	1.50

3.4 Structural Branch

Each structural encoder is a two-layer GATv2 network [5]. The first layer uses eight heads of width $64/8 = 8$, concatenated to 64; the second uses a single head of width 64. Both layers receive the 39-dimensional encoded edge vector as an attention input and add no self-loops. Each layer applies, in order, the GATv2 convolution, batch normalisation, an ELU nonlinearity, and a residual connection; dropout at $p = 0.2$ is applied before the residual in the first layer only. The first layer’s residual passes through a linear projection from the 10 input channels to 64; the second’s is an identity. The final node embedding is 64-dimensional.

We use GATv2 rather than GAT because of the static-attention limitation [5]: in GAT the ranking a node induces over its neighbours can be independent of the query node, so attention degenerates into a global neighbour scoring. GATv2 applies its attention MLP after the concatenation of both endpoints, which restores query-dependent ranking. On a graph with 49 nodes and heavy parallel-edge structure, that distinction is not cosmetic.

Three encoders run in parallel — *temporal*, *content*, and *behavioural*. They are architecturally identical and differ only in a fixed diagonal reweighting of the encoded edge vector, given in Table 1. The protocol weight is broadcast across all 4 protocol-embedding dimensions and the port weight across all 32, so each variant keeps the same relative emphasis it would have had on the five raw columns. The reweighted edge vector is used both inside the attention computation and in the edge representation that follows, so a variant’s emphasis shapes its message passing as well as its readout.

Within a variant, the representation of a directed edge $e = (u, v)$ concatenates the two endpoint embeddings, their elementwise product, their elementwise absolute difference, and the variant’s reweighted edge vector $\tilde{\mathbf{x}}_e$:

$$\mathbf{z}_e = [\mathbf{h}_u, \mathbf{h}_v, \mathbf{h}_u \odot \mathbf{h}_v, |\mathbf{h}_u - \mathbf{h}_v|, \tilde{\mathbf{x}}_e] \in \mathbb{R}^{4 \cdot 64 + 39} = \mathbb{R}^{295}.$$

The product and absolute-difference terms matter here. Endpoint concatenation

alone cannot separate parallel edges, and 58.7% of NF-UNSW-NB15 edges share an IP pair with at least one other edge; the elementwise terms are symmetric functions of the endpoints and do not fix this on their own, but $\tilde{\mathbf{x}}_e$ does, by carrying the edge’s own features into the classifier.

Each variant projects $\mathbf{z}_e$ to 64 dimensions through a linear layer and ReLU. The three projections are concatenated and passed through a linear layer to 64, ReLU, and dropout, yielding the fused 64-dimensional edge embedding consumed by every downstream stage; a final linear layer emits class logits. Each variant additionally carries its own classification head — a two-layer MLP over $\mathbf{z}_e$ with a 64-unit hidden layer, ReLU and dropout — whose loss is added to the objective with weight 0.30. The auxiliary term keeps each variant directly supervised rather than letting the fusion layer absorb all gradient.

3.5 Semantic Branch

The second modality describes each edge in controlled natural language. The continuous edge quantities are first discretised into five rank-based quantile bands, computed globally across all edges, and the band label rather than the raw number enters the sentence. A template then renders endpoints, connection frequency, transferred volume, per-flow byte magnitude, duration band, protocol name, and a coarse port role (web, remote-access, well-known, registered, or ephemeral) into one sentence per edge, of the form: *“Observed traffic from source ... to destination ... with ... connection frequency, transferring ... as ..., ..., over ..., to a ... port ...”*

Sentences are encoded by CySecBERT [3], a BERT-class encoder domain-adapted to cybersecurity text. It is frozen throughout; no gradient reaches it. Inputs are tokenised with padding and truncation at 128 tokens, and the final hidden states are mean-pooled with padding positions masked out, giving one 768-dimensional vector per edge. We use it strictly as a sentence encoder. It is not prompted, not asked to reason, and not generative, and we avoid describing it as an LLM for that reason.

Label freedom is enforced in code, not by convention. Before any sentence reaches the encoder, an assertion scans every generated sentence for every attack term present in the dataset and raises on the first match, naming the offending index and sentence. The pipeline halts. This is what makes the semantic branch an independent view of the edge rather than a relabelling of it.

3.6 Adaptive Gated Attention Fusion

AGAF projects each modality into a shared 128-dimensional space through a linear layer, batch normalisation and ReLU — 64 to 128 for the structural embedding, 768 to 128 for the semantic one. A gate consumes the concatenated projections and emits two logits, softmaxed at temperature 1.0 into weights (α_g, α_l) that convexly combine them:

$$\mathbf{f}_e = \alpha_g(e) \mathbf{g}_e + \alpha_l(e) \mathbf{l}_e, \quad \alpha_g(e) + \alpha_l(e) = 1.$$

The weights are per-edge, so the balance between structure and semantics is free to vary across the graph rather than being fixed at concatenation time. A two-layer MLP over $\mathbf{f}_e$ produces class logits. Both inputs are standardised before entering the module; the scalars are held in the trainer and persisted separately, so the module’s `state_dict` stays free of fitted statistics.

One constraint on this stage is load-bearing. AGAF consumes *per-fold out-of-fold* structural embeddings only. It never receives embeddings from a GNN fitted on all edges, because such a model has already seen the test edges’ labels, and those labels would reach the fusion classifier through its own input features — a leak that no split downstream of it could detect. For each held-out fold, the structural embeddings supplied to AGAF come from a model that never saw that fold’s labels. Earlier runs in this project violated this rule, and the reproduction entry point now enforces it.

3.7 Uncertainty-Guided Semantic Feedback

The feedback stage routes semantic information to the edges the structural model is least sure about, and only there.

Selection. For each edge we compute the Shannon entropy, in nats, of the GNN’s predictive distribution, and flag the edges above the $(1 - k/100)$ quantile. The threshold is calibrated once and reused, so the flagged set is a fixed-size top- $k\%$ rather than a moving target. We select k on validation folds only: $k = 31.0$ on NF-UNSW-NB15, $k = 25.0$ on NF-ToN-IoT.

Consultation. The semantic consultant is a whitened prototype scorer. Within each fold, we centre the training-fold semantic embeddings, fit a ZCA whitener by eigendecomposition of their covariance with a floor on small eigenvalues — necessary because the training fold has fewer rows than the 768 embedding dimensions — and take the class-wise mean in the whitened space as that class’s prototype. At inference an edge is whitened with the same fold-local transform and scored by cosine similarity against the

prototypes. Whitening is not incidental: the earlier L2-normalise-average-renormalise scheme collapsed per-class spread until random and real advice became indistinguishable. Every rung uses this same scorer; none uses a trained supervised head, which would make the loop echo a classifier it already contains.

Gating. Being uncertain in the structural view does not make an edge a good candidate for semantic advice. Among the flagged edges we keep only the most confident half by the consultant’s own maximum class probability, a semantic confidence fraction of 0.50. The effective feedback rate is thus 15.5% of edges on NF-UNSW-NB15 and 12.5% on NF-ToN-IoT.

Injection. Accepted advice is injected into the *edge representation* consumed by the next graph update. The consultant’s class logits are projected linearly to the 39-dimensional edge width, scaled by a learned strength, written into a full-graph bias tensor that is exactly zero on unflagged edges, and added to the reweighted edge vector:

$$\tilde{\mathbf{x}}_e \leftarrow \tilde{\mathbf{x}}_e + s \cdot \mathbf{b}_e, \quad \mathbf{b}_e = \mathbf{0} \text{ for unflagged } e,$$

with injection scale $s = 2.0$ on NF-UNSW-NB15 and $s = 20.0$ on NF-ToN-IoT, both selected on validation folds. Because $\tilde{\mathbf{x}}_e$ enters both attention and readout, the injected signal propagates structurally rather than only shifting a final logit. The scale is re-selected whenever the edge encoding or the injection path changes, since it is expressed in the units of that encoding.

Iteration. Predictions are recomputed, entropies re-derived, and the cycle repeats. We measure *churn*, the fraction of edges whose predicted class changed since the previous iteration, and stop when churn falls below 0.01 or after three iterations, whichever comes first.

An alternative that was implemented and measured. We also built a variant that adds the projected semantic signal to GATv2’s unnormalised attention logits, before the per-edge softmax — injecting into the attention mechanism rather than the edge representation. It is zero-initialised, so an untrained bias reproduces stock GATv2 exactly. It is evaluated under the identical protocol and reported in Section ?? as a negative result. We retain it in the paper because the two variants isolate *where* in the network a semantic signal must land in order to have any effect at all, which is a question the positive result alone does not answer.

3.8 Baseline Protocol

We compare against re-implementations of two published graph-based intrusion detectors, E-GraphSAGE [12] and TE-G-SAGE [?], retrained on the same aggregated edges, the same labels and the same folds. Each baseline keeps its own published featurisation rather than ours — E-GraphSAGE standard-scales all five NetFlow columns; TE-G-SAGE applies log1p, standard scaling, correlation pruning at 0.995, and one-hot categoricals with rare-category folding.

Each baseline is run in three configurations. `as_published` reproduces the released implementation. `refit` re-selects capacity and schedule on our validation folds only. `plus_node_features` is deliberately non-faithful: it hands the baseline our ten centrality node features, isolating how much of any gap is representation rather than architecture. We label it as non-faithful wherever it appears; it is not E-GraphSAGE and it is not TE-G-SAGE.

Five deviations from the published specifications were unavoidable at this graph scale, and we record them rather than absorb them. (D1) TE-G-SAGE’s chronological 60/30/10 split cannot be reconstructed, because aggregation collapses per-flow timestamps; our stratified folds are used for all models. (D2) TE-G-SAGE’s neighbour fanout of (25, 15) and batch size of 4096 both exceed the entire graph, so training is full-neighbourhood and full-batch. (D3) Both released implementations train for a fixed epoch count with no validation split and no early stopping — 4999 epochs for E-GraphSAGE, 20 for TE-G-SAGE — which is incompatible with a protocol that selects on validation folds; baselines are given 300 epochs with patience 25 on validation macro-F1 and the best state restored. (D4) TE-G-SAGE’s `rare_min_freq` of 50 removes almost every port category at our scale; it is applied as published in `as_published` and swept in `refit`. (D5) E-GraphSAGE’s published Eq. 4 aggregates edge features alone, but the authors’ released code builds the message as $W_{\text{msg}}([\mathbf{h}_u || \mathbf{e}_{uv}])$; we follow the code, since the published numbers came from it.

Two asymmetries remain and should be weighed when reading the comparison. Our rungs train under a focal objective with patience 40, whereas the baselines use class-weighted cross-entropy with patience 25, each keeping its own published loss. And E-GraphSAGE classifies an edge from $[\mathbf{h}_u || \mathbf{h}_v]$ alone, so parallel edges between one IP pair are mathematically indistinguishable to it — 58.7% of NF-UNSW-NB15 edges and 7.9% of NF-ToN-IoT edges fall in such groups. That is an interaction between its architecture and our aggregated representation, not evidence that E-GraphSAGE is a weak detector, and we do not read it as one.

3.9 Evaluation and Training Protocol

We use stratified five-fold cross-validation over edges with partition seed 42, represented as boolean masks. Within each fold the held-out fifth is the test partition, and the remaining four-fifths are split 75/25 into training and validation with seed 42 + fold index, giving approximately 60/20/20. Test masks are used once, for prediction.

The primary metric is pooled out-of-fold macro-F1: each edge is predicted only while held out, and the five sets of out-of-fold predictions are pooled before a single macro-F1 is computed. Model selection uses validation folds exclusively. The uncertainty threshold k and the injection scale were selected per dataset by a validation sweep ($k \in [15, 35]$; $\text{scale} \in \{0.5, 1, 2, 5, 10, 20\}$) averaged across three seeds (42, 1, 2); the remaining architectural hyperparameters were fixed a priori and not swept. We note this as a limitation: k was selected on rotating validation folds rather than by nested cross-validation, so the final estimate is not fully unbiased with respect to that one choice.

Every rung is trained at three training seeds (42, 1, 2) with the fold partition held fixed at the seed-42 split, so all seeds score the same held-out edges and differ only in initialisation, dropout and data order. We report the mean and standard deviation of pooled out-of-fold macro-F1 over the three seeds; the standard deviation therefore measures training-seed variance at a fixed partition, not total uncertainty, since fold-partition variance is not measured. Pairwise differences between rungs use a two-level bootstrap with 2,000 iterations: each iteration draws a training seed independently for each side and resamples edges with replacement, so the interval reflects both sources of variation. A seed-matched bootstrap (seed 42 against 42, 1 against 1, 2 against 2) is reported as a secondary check. The decision rule was fixed before the runs: a difference is called *separated* only if its two-level 95% interval excludes zero, and *not separated* otherwise; we do not describe point-estimate orderings as significant. The semantic rung is deterministic given the folds and has zero seed variance, so it enters comparisons as a fixed side. Comparisons against re-trained published baselines use the same procedure with three baseline seeds on the other side.

Training uses AdamW with learning rate 10^{-3} and weight decay 5×10^{-4} , at most 300 epochs, early stopping with patience 40 on validation macro-F1 with the best state restored, and gradient clipping at norm 1.0. The objective is focal loss with $\gamma = 2.0$ and α set to inverse-frequency class weights computed on the training fold, normalised to sum to the class count; classes absent from a training fold receive weight zero. Focal loss down-weights already-correct examples, which matters when rare attack classes hold 12–35 edges against a benign majority. We do not additionally oversample: combining oversampling with these weights double-corrects the imbalance and collapses the fusion

stage.

Every run exports `OMP_NUM_THREADS=1`. Graph-attention message passing sums neighbour contributions in thread-completion order, and floating-point addition is not associative, so thread count alone moves pooled macro-F1 by roughly 0.012 at a fixed seed — larger than several of the effects this paper reports. Semantic encoding is likewise pinned to a single device, because MPS and CPU produce slightly different embeddings and every downstream number inherits the difference.

Two NF-ToN-IoT classes are smaller than the five-fold minimum: `dos` with 4 edges and `ransomware` with 3. They remain in the graph and continue to participate in message passing, but are excluded from the reported metric, so NF-ToN-IoT macro-F1 is computed over 8 classes and NF-UNSW-NB15 over 10. The two are therefore not comparable in absolute level, and we compare only the shape of the ladder across datasets, never the values.

References

- [1] GCN-2-Former: Hybrid intrusion detection based on dynamic spatial-temporal GNN in in-vehicle networks. *Scientific Reports*, 2025. s41598-025-18401-3; author list to be completed.
- [2] John Arevalo, Thamar Solorio, Manuel Montes y Gómez, and Fabio A. González. Gated multimodal units for information fusion, 2017. arXiv:1702.01992.
- [3] Markus Bayer, Philipp Kuehn, Ramin Shanehsaz, and Christian Reuter. Cy-SecBERT: A domain-adapted language model for the cybersecurity domain, 2022. arXiv:2212.02974.
- [4] Azadeh Beiranvand and Seyed Mehdi Vahidipour. BiGTex: Integrating structural and semantic signals in text-attributed graphs, 2025. arXiv:2504.12474.
- [5] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022. doi: 10.48550/arXiv.2105.14491. Verified via Scite 2026-09-05; no editorial notices.
- [6] Yasir Ali Farrukh, Syed Wali, Irfan Khan, and Nathaniel D. Bastian. XG-NID: Dual-modality network intrusion detection using a heterogeneous graph neural network and large language model, 2024. arXiv:2408.16021.
- [7] Guo et al. DET: A dual-encoding transformer for relational graph embedding. In *LREC-COLING 2024*, pages 4685–4696, 2024. Author list to be completed.
- [8] Huang et al. BSTFNet: Encrypted malicious traffic classification integrating global semantic and spatiotemporal features. *Computers, Materials & Continua*, 78(3): 3929–3951, 2024. Author list to be completed.
- [9] Lee et al. CAST: Cross-attention based multimodal fusion of structure and text for materials property prediction, 2025. arXiv:2502.06836; author list to be completed.
- [10] Haolin Li et al. RAGFormer: Fraud detection with binding global and local relational interaction, 2024. arXiv:2402.17472; author list to be completed.
- [11] Yuxiao Lin, Yuxian Meng, Xiaofei Sun, Qinghong Han, Kun Kuang, Jiwei Li, and Fei Wu. BertGCN: Transductive text classification by combining GNN and BERT. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021.

- [12] Wai Weng Lo, Siamak Layeghy, Mohanad Sarhan, Marcus Gallagher, and Marius Portmann. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, pages 1–9, 2022. doi: 10.1109/NOMS54207.2022.9789878. Verified via Scite 2026-09-05; no editorial notices. Code: github.com/waimorris/E-GraphSAGE.
- [13] Donald Loveland, Xinyi Yang, and Danai Koutra. Glance for context: Learning when to leverage LLMs for node-aware GNN-LLM fusion, 2025. arXiv:2510.10849.
- [14] Hejamadi Rama Moorthy, N. J. Avinash, N. S. Krishnaraj Rao, K. R. Raghunandan, Radhakrishna Dodmane, Jeremy Joseph Blum, and Lubna A. Gabralla. Dual stream graph augmented transformer model integrating BERT and GNNs for context aware fake news detection. *Scientific Reports*, 15:25436, 2025. doi: 10.1038/s41598-025-05586-w.
- [15] Nour Moustafa. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustainable Cities and Society*, 72:102994, 2021. doi: 10.1016/j.scs.2021.102994. Verified via Scite 2026-09-05; no editorial notices.
- [16] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, 2015. doi: 10.1109/MilCIS.2015.7348942.
- [17] Yiran Qiao, Xiang Ao, and Yang Liu. LOGIN: A large language model consulted graph neural network training framework, 2024. arXiv:2405.13902.
- [18] Mohanad Sarhan, Siamak Layeghy, Nour Moustafa, and Marius Portmann. Net-flow datasets for machine learning-based network intrusion detection systems. In *Big Data Technologies and Applications*, pages 117–135. Springer International Publishing, 2021. doi: 10.1007/978-3-030-72802-1_9.
- [19] Karen Scarfone and Peter Mell. Guide to intrusion detection and prevention systems (IDPS). Technical Report NIST Special Publication 800-94, National Institute of Standards and Technology, 2007.
- [20] Aarush Sinha. GMLM: Bridging graph neural networks and language models for heterophilic node classification, 2025. arXiv:2503.05763.
- [21] Bishal Thapaliya, Yu Wang, and Jingyu Li. Semantic refinement with LLMs for graph representations, 2025. arXiv:2512.21106.

- [UNVERIFIED] 2025)]tegsage2025 [AUTHORS UNVERIFIED]. TE-G-SAGE: Edge-aware GraphSAGE with explainability for network intrusion detection. *IoT*, 6(4): 165, 2025.
- [23] Yiran Wang, Xiang Gao, and Sarthak Kharel. Are LLMs better GNN helpers? rethinking robust graph learning under deficiencies with iterative refinement, 2025. arXiv:2510.01910; system named RoGRAD.
- [24] Xiong, Wan, and Ren. Two-branch intrusion detection method based on fusion of deep semantic and statistical features. *Computers, Materials & Continua*, 87(3), 2026. Author initials to be completed.
- [25] Yang, Wang, et al. GL-Fusion: Rethinking the combination of graph neural network and large language model, 2024. arXiv:2412.06849; author list to be completed.
- [26] Zhonghao Zhan, Huichi Zhou, and Hamed Haddadi. Poster: Enhancing GNN robustness for network intrusion detection via agent-based analysis, 2025. arXiv:2506.20806.